Branch: MCA (Data Science)	**Semester:** 2
Student Name: Adarsh Mishra	**UID:** 25MCD10065
Subject Name: Technical Training - I Lab	**Subject Code**: 25CAP-652
Section/Group: 25MCD KAR-1	**Date of Performance:** 04 April,2026

EXPERIMENT – 07

Aim

To implement different types of **JOIN operations in PostgreSQL** and retrieve meaningful data from a relational database consisting of Students, Courses, Enrollments, and Departments tables.

Tools Used

- PostgreSQL

Objectives

- Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)
  1. Write queries to list students with their enrolled courses (INNER JOIN).
  2. Find students not enrolled in any course (LEFT JOIN).
  3. Display all courses with or without enrolled students (RIGHT JOIN).
  4. Show students with department info using SELF JOIN or multiple joins.
  5. Display all possible student-course combinations (CROSS JOIN). (Oracle, SAP, IBM, Microsoft)

Theory

In relational databases, data is stored in multiple tables to avoid redundancy. To combine data from these tables, **SQL JOIN operations** are used.

Types of JOINs:

- **INNER JOIN**
  Returns only the records that have matching values in both tables.
- **LEFT JOIN (LEFT OUTER JOIN)**
  Returns all records from the left table and matched records from the right table. If no match exists, NULL values are returned.
- **RIGHT JOIN (RIGHT OUTER JOIN)**
  Returns all records from the right table and matched records from the left table.
- **CROSS JOIN**
  Returns all possible combinations of rows from both tables (Cartesian product).
- **MULTIPLE JOIN**
  Used to combine more than two tables in a single query.

.

Experiment Steps:

Step 1: Create Tables

- Create Departments, Students, Courses, and Enrollments tables
- Define primary keys and foreign keys

Step 2: Insert Data

- Insert sample data into all tables
- Ensure some students are not enrolled (for LEFT JOIN testing)

Step 3: Execute INNER JOIN

- Retrieve students along with their enrolled courses
- Verify only matching records are displayed

Step 4: Execute LEFT JOIN

- Find students who are not enrolled in any course
- Check for NULL values in unmatched rows

Step 5: Execute RIGHT JOIN

- Display all courses including those without students
- Ensure unmatched courses are included

Step 6: Execute MULTIPLE JOIN

- Combine Students and Departments tables
- Display student names along with department details

Step 7: Execute CROSS JOIN

- Generate all possible student-course combinations
- Observe Cartesian product behavior

Step 8: Verify Results

- Compare outputs with expected results
- Ensure correctness of JOIN operations

Query:

```
CREATE TABLE Departments (
   department_id SERIAL PRIMARY KEY,
   department_name VARCHAR(100) NOT NULL
);

CREATE TABLE Students (
   student_id SERIAL PRIMARY KEY,
   name VARCHAR(100) NOT NULL,
   department_id INT,
   FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);
```

```
CREATE TABLE Courses (
   course_id SERIAL PRIMARY KEY,
   course_name VARCHAR(100) NOT NULL
);

CREATE TABLE Enrollments (
   enrollment_id SERIAL PRIMARY KEY,
   student_id INT,
   course_id INT,
   FOREIGN KEY (student_id) REFERENCES Students(student_id),
   FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);


INSERT INTO Departments (department_name) VALUES
('Computer Science'),
('Mechanical'),
('Electronics');

INSERT INTO Students (name, department_id) VALUES
('Adarsh', 1),
('Ritesh', 2),
('Kunal', 1),
('Sameer', 3),
('Chaitanya', NULL);

INSERT INTO Courses (course_name) VALUES
('DBMS'),
('Operating Systems'),
('Thermodynamics'),
('Digital Electronics');

INSERT INTO Enrollments (student_id, course_id) VALUES
(1, 1),
(1, 2),
(2, 3),
(3, 1),
(4, 4);
```

--Tables

```
select * from departments;
select * from students;
select * from courses;
select * from enrollments;
```

Output:

	department_id [PK] integer	department_name character varying (100)
1	1	Computer Science
2	2	Mechanical
3	3	Electronics

	student_id [PK] integer	name character varying (100)	department_id integer
1	1	Adarsh	1
2	2	Ritesh	2
3	3	Kunal	1
4	4	Sameer	3
5	5	Chaitanya	[null]

	course_id [PK] integer	course_name character varying (100)
1	1	DBMS
2	2	Operating Systems
3	3	Thermodynamics
4	4	Digital Electronics

	enrollment_id [PK] integer	student_id integer	course_id integer
1	1	1	1
2	2	1	2
3	3	2	3
4	4	3	1
5	5	4	4

--Inner Join

```
SELECT s.student_id, s.name, c.course_name
FROM Students s
INNER JOIN Enrollments e
    ON s.student_id = e.student_id
INNER JOIN Courses c
    ON e.course_id = c.course_id;
```

Output:

	student_id integer	name character varying (100)	course_name character varying (100)
1	1	Adarsh	DBMS
2	1	Adarsh	Operating Systems
3	2	Ritesh	Thermodynamics
4	3	Kunal	DBMS
5	4	Sameer	Digital Electronics

-- Left Join

```
SELECT s.student_id, s.name
FROM Students s
LEFT JOIN Enrollments e
    ON s.student_id = e.student_id
WHERE e.student_id IS NULL;
```

Output:

	student_id [PK] integer	name character varying (100)
1	5	Chaitanya

-- Right Join

```
SELECT c.course_id, c.course_name, s.name
FROM Students s
RIGHT JOIN Enrollments e
    ON s.student_id = e.student_id
```

```
RIGHT JOIN Courses c
    ON e.course_id = c.course_id;
```

Output:

	course_id integer	course_name character varying (100)	name character varying (100)
1	1	DBMS	Adarsh
2	2	Operating Systems	Adarsh
3	3	Thermodynamics	Ritesh
4	1	DBMS	Kunal
5	4	Digital Electronics	Sameer

-- Multiple Join

```
SELECT s.student_id, s.name, d.department_name
FROM Students s
INNER JOIN Departments d
        ON s.department_id = d.department_id;
```

Output:

	student_id integer	name character varying (100)	department_name character varying (100)
1	1	Adarsh	Computer Science
2	2	Ritesh	Mechanical
3	3	Kunal	Computer Science
4	4	Sameer	Electronics

-- Cross Join

```
SELECT s.name, c.course_name
FROM Students s
CROSS JOIN Courses c;
```

Output:

	name character varying (100)	course_name character varying (100)
1	Adarsh	DBMS
2	Adarsh	Operating Systems
3	Adarsh	Thermodynamics
4	Adarsh	Digital Electronics
5	Ritesh	DBMS
6	Ritesh	Operating Systems
7	Ritesh	Thermodynamics
8	Ritesh	Digital Electronics
9	Kunal	DBMS
10	Kunal	Operating Systems
11	Kunal	Thermodynamics
12	Kunal	Digital Electronics
13	Sameer	DBMS
14	Sameer	Operating Systems
15	Sameer	Thermodynamics
16	Sameer	Digital Electronics
17	Chaitanya	DBMS
18	Chaitanya	Operating Systems
19	Chaitanya	Thermodynamics
20	Chaitanya	Digital Electronics

Learning Outcomes:

- Understand the concept of **relational databases** and table relationships.
- Identify and differentiate between various types of **SQL JOIN operations** such as INNER JOIN, LEFT JOIN, RIGHT JOIN, and CROSS JOIN.
- Apply JOIN operations to retrieve data from **multiple related tables**.
- Use **foreign keys and primary keys** to establish relationships between tables.
- Analyze query results to understand how different JOINs affect output.
- Implement **multiple table joins** to solve real-world database problems.
- Interpret and validate query outputs for correctness.